

AeroNet Lite

Autonomous Drone Delivery Simulation with CSP, Fleet Planning, A* Routing, Real-Time Replanning, Demand Forecasting, and Anomaly Detection

Course: Artificial Intelligence

Submitted to: Mam Mahnoor Tariq

University: FAST-NUCES Islamabad

Program: BS Data Science

Semester: Spring 2026

Student Name	Section	Roll Number
Hammad ur Rehman	DS_C	23i-2504
Muhammad Abdullah	DS_A	23i-2655
Abdullah	DS_A	23i-2585
Ali Waseem	DS_A	23i-2630
Ahmed Bilal	DS_C	23i-2581

Abstract

AeroNet Lite is a compact AI-based drone delivery simulator that models a city as a 10 x 10 grid. The project integrates constraint satisfaction for city validation, optimization for fleet selection, A* search for route planning, real-time rerouting after no-fly disruptions, and machine learning for demand forecasting and anomaly classification. This updated report also includes the executed notebook results from the Amazon Delivery demand forecasting notebook and the drone anomaly classifier notebook.

1. Project Overview

The objective of AeroNet Lite is to demonstrate how multiple AI techniques can work together in a realistic but manageable drone-delivery scenario. Instead of building a full city-scale autonomous system, the simulation uses a 10 x 10 grid where each cell contains a zone type, demand value, facility flags, and no-fly status. This makes the project explainable for viva while still covering multiple Artificial Intelligence concepts.

The implemented dashboard supports interactive editing of grid cells, validation of operating constraints, fleet planning under a fixed budget, A* path planning, rerouting after disruption, machine learning prediction, anomaly detection, and a 20-step integrated demo.



Shared state: city grid, no-fly flags, fleet, routes, event log, and ML outputs.

2. Dataset and ML Strategy

For demand forecasting, the external dataset used in the notebook is the Kaggle Amazon Delivery Dataset. The uploaded notebook found amazon_delivery.csv with 43,739 raw records and 16 columns. After grouping delivery records into demand units by time and operating context, the prepared training table contained 379 rows and 10 columns. For anomaly classification, synthetic drone telemetry is used because the classes are generated from explainable fault rules.

ML Task	Dataset / Source	Final Target	Models Compared	Metric
Demand Forecasting	Amazon Delivery Dataset	Grouped delivery demand count	Linear Regression, Random Forest	MAE, RMSE, R2
Anomaly Classification	Synthetic drone telemetry	Normal, battery, route, sensor classes	Decision Tree, Random Forest	Accuracy, confusion matrix

3. Module Summary

Module	AI Technique	Purpose
Grid and Layout Validator	Constraint Satisfaction Problem	Checks safety and coverage rules on the city layout
Fleet Selector	Brute force / Genetic Algorithm	Chooses light and heavy drones under budget
Delivery Path Planner	A* Search	Finds shortest safe route avoiding no-fly cells
Disruption Handler	Replanning / Optimization	Reroutes drones after dynamic no-fly activation
ML Pipeline	Regression + Classification	Forecasts demand and detects flight anomalies

4. Interactive City Dashboard

The main dashboard is the control center of the system. It shows the complete city grid, demand heatmap, hub count, no-fly cell count, and total estimated demand. The side panel allows the user to reset the city, load an intentionally invalid grid for CSP testing, and edit one grid cell by changing its zone, hub status, charging pad, and no-fly flag.

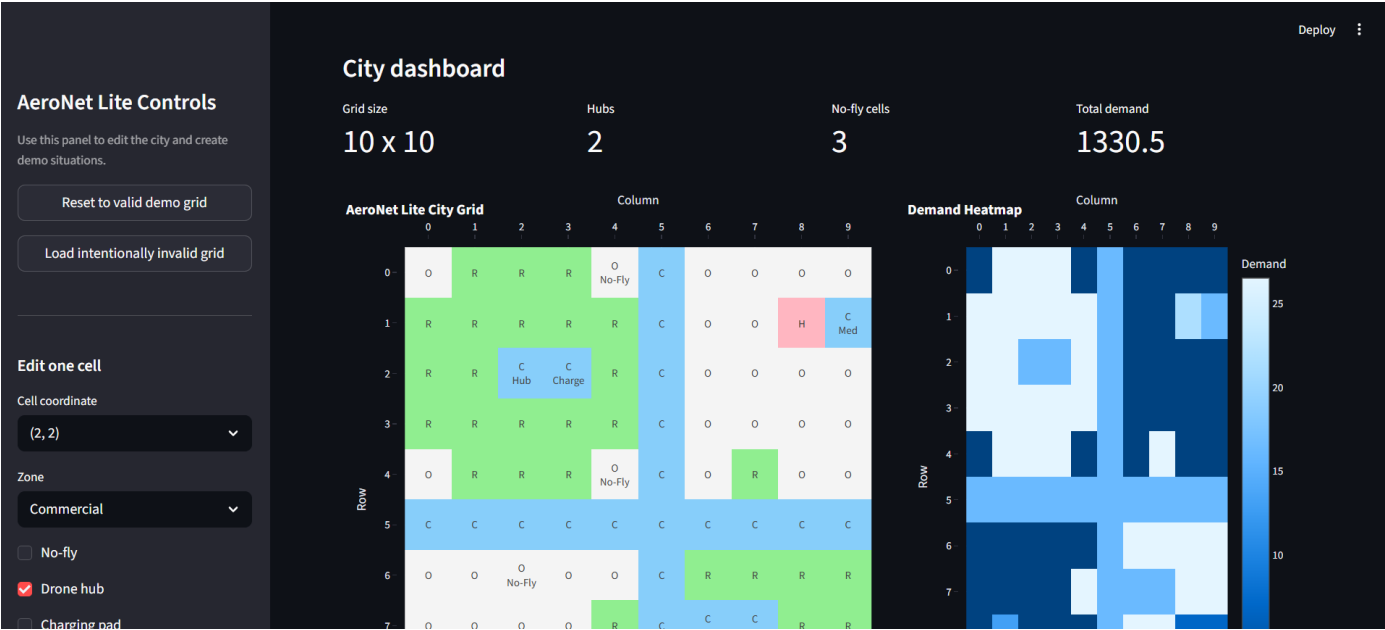


Figure 1: AeroNet Lite dashboard showing grid controls, city map, demand heatmap, hubs, no-fly cells, and total demand.

Implementation point: each grid cell acts as the shared state object for all modules. CSP reads grid rules, A* reads no-fly status and movement cost, fleet planning reads demand, and the simulation updates event logs and route states.

5. CSP Layout Validation and Fleet Selection

The CSP validator checks four operating constraints: industrial safety buffer, residential hub coverage, hub charging access, and medical pickup access. The demo grid passes all four rules, so route planning and simulation can proceed. The fleet selector ranks drone combinations using a score based on demand coverage and budget usage.

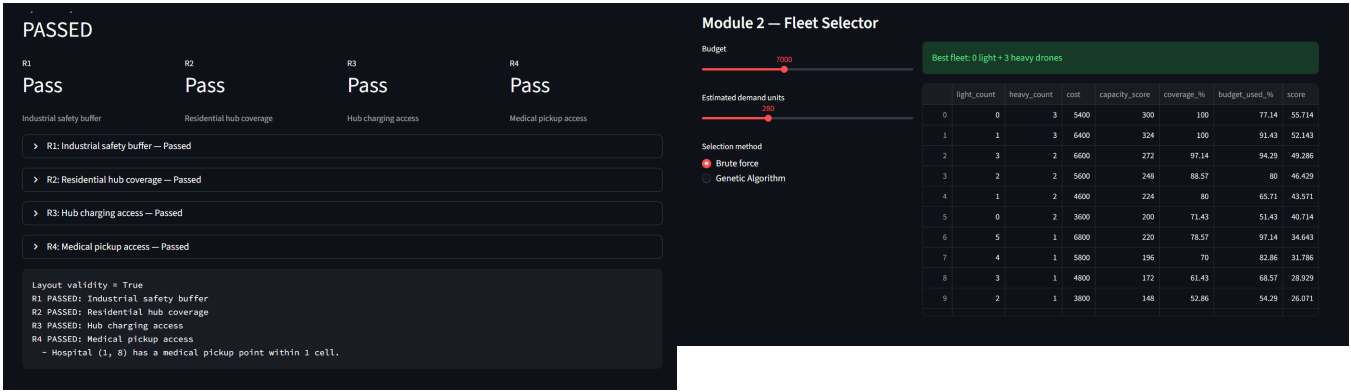


Figure 2: CSP validation result and fleet selector output. The valid demo grid passes all four rules; fleet selection ranks drone combinations by coverage and budget efficiency.

Decision / Rule	Observed Result	Meaning
CSP R1-R4	Passed	City layout is safe enough for operation
Best fleet	0 light + 3 heavy drones	Full demand coverage under budget
Budget used	5400 / 7000 = 77.14%	Efficient solution without spending full budget
Coverage	100%	Selected fleet can serve estimated demand

6. A* Delivery Route Planning

The path planner uses A* search on grid coordinates. The drone can move up, down, left, and right. No-fly cells are skipped completely. Manhattan distance is used as the heuristic because it is admissible for a four-direction grid. The route shown in the dashboard avoids blocked cells and follows safe grid movement.

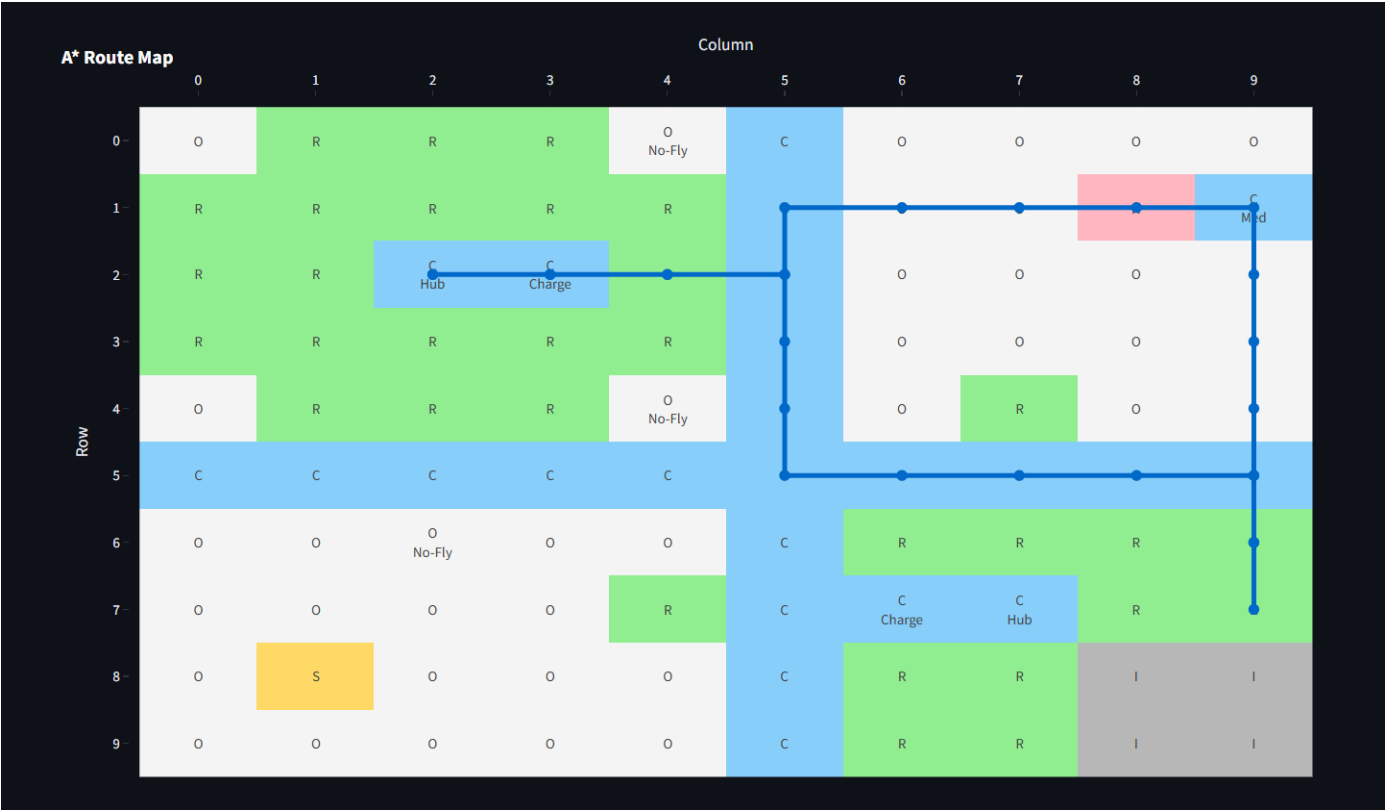


Figure 3: A* route map. The route avoids no-fly cells and connects delivery points through safe grid movement.

A* Component	Implementation Choice
State	Grid coordinate (row, column)
Actions	Move up, down, left, right
Path Cost	Normal grid movement with optional commercial corridor preference
Blocked Nodes	Cells where no_fly = True
Heuristic	Manhattan distance

7. Real-Time Disruption Handling

The disruption handler demonstrates real-time replanning. A user selects the drone current position, the next target, and a cell to block. The system first calculates the original route, then marks the selected cell as no-fly and reruns A* from the current drone position to the same target. If a safe route exists, the drone is rerouted; otherwise, the delivery can be delayed or failed.

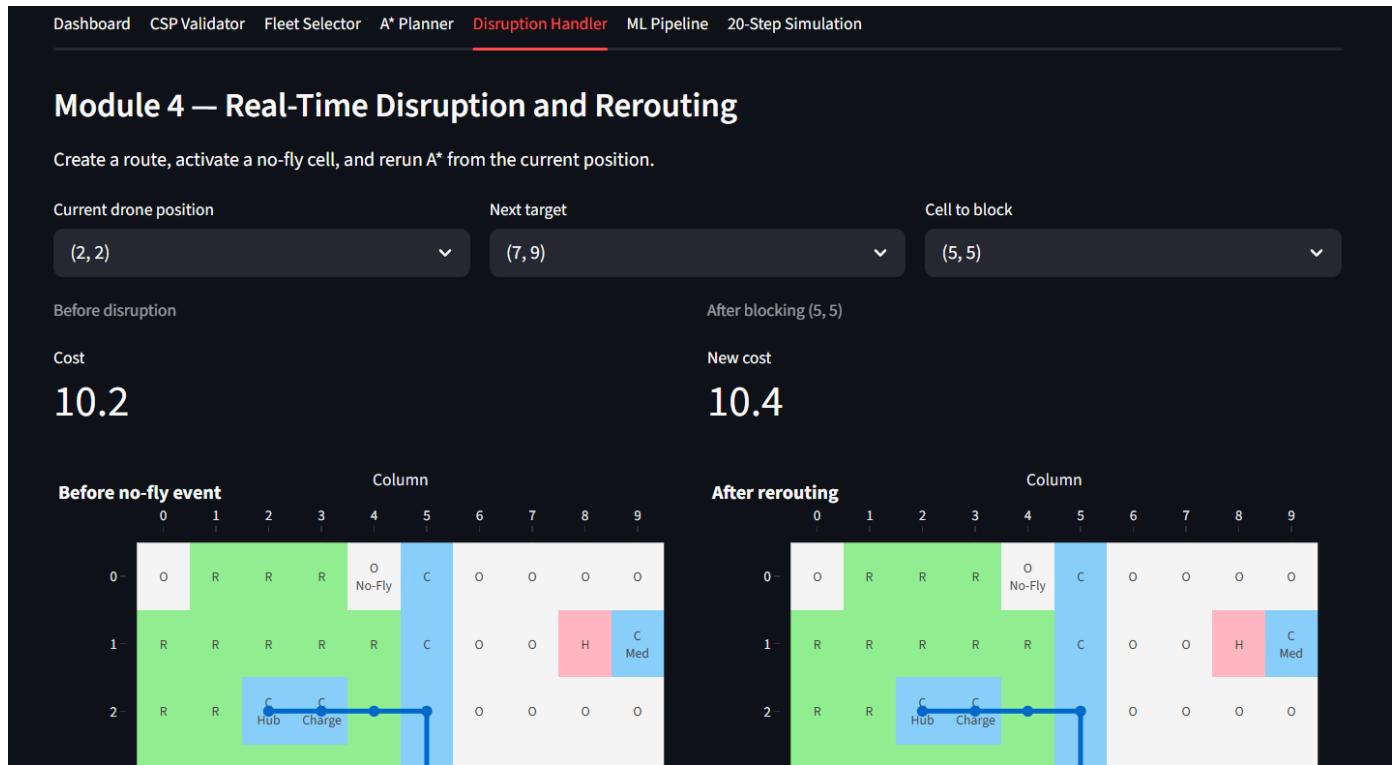


Figure 4: Disruption handler before and after no-fly activation. The route cost changes from 10.2 to 10.4 after rerouting.

Defense point: this module proves the system is not a static route drawer. It reacts to a live environmental change by updating the grid and recalculating the route using the same A* engine.

8. Notebook Result: Amazon Delivery Demand Forecasting

The demand forecasting notebook was executed with amazon_delivery.csv. The original dataset contained 43,739 delivery records with 16 columns. After feature engineering and grouping into demand units, the prepared demand table contained 379 rows and 10 columns. Random Forest performed much better than Linear Regression, which is expected because delivery demand is nonlinear and strongly affected by time and context.

Random Forest Best model Lowest RMSE in notebook	10.619 MAE Average absolute demand error	17.399 RMSE Penalizes larger errors	0.982 R2 Explained variance on test split
---	---	--	--

Model	MAE	RMSE	R2
Random Forest	10.619	17.399	0.982
Linear Regression	59.621	73.477	0.680

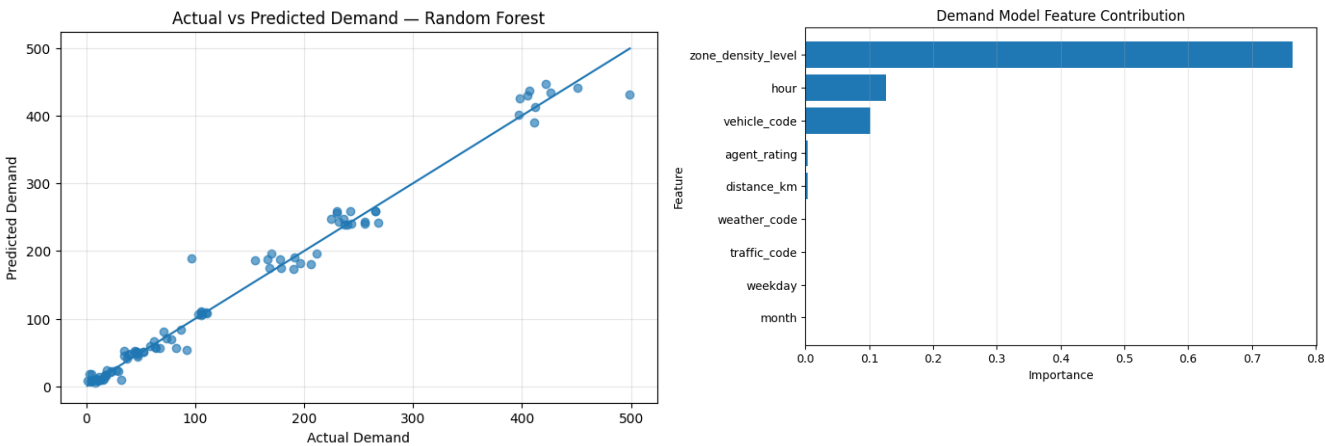


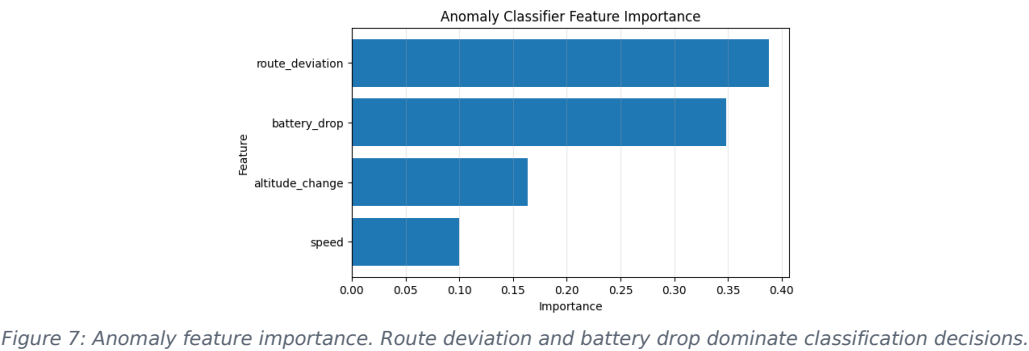
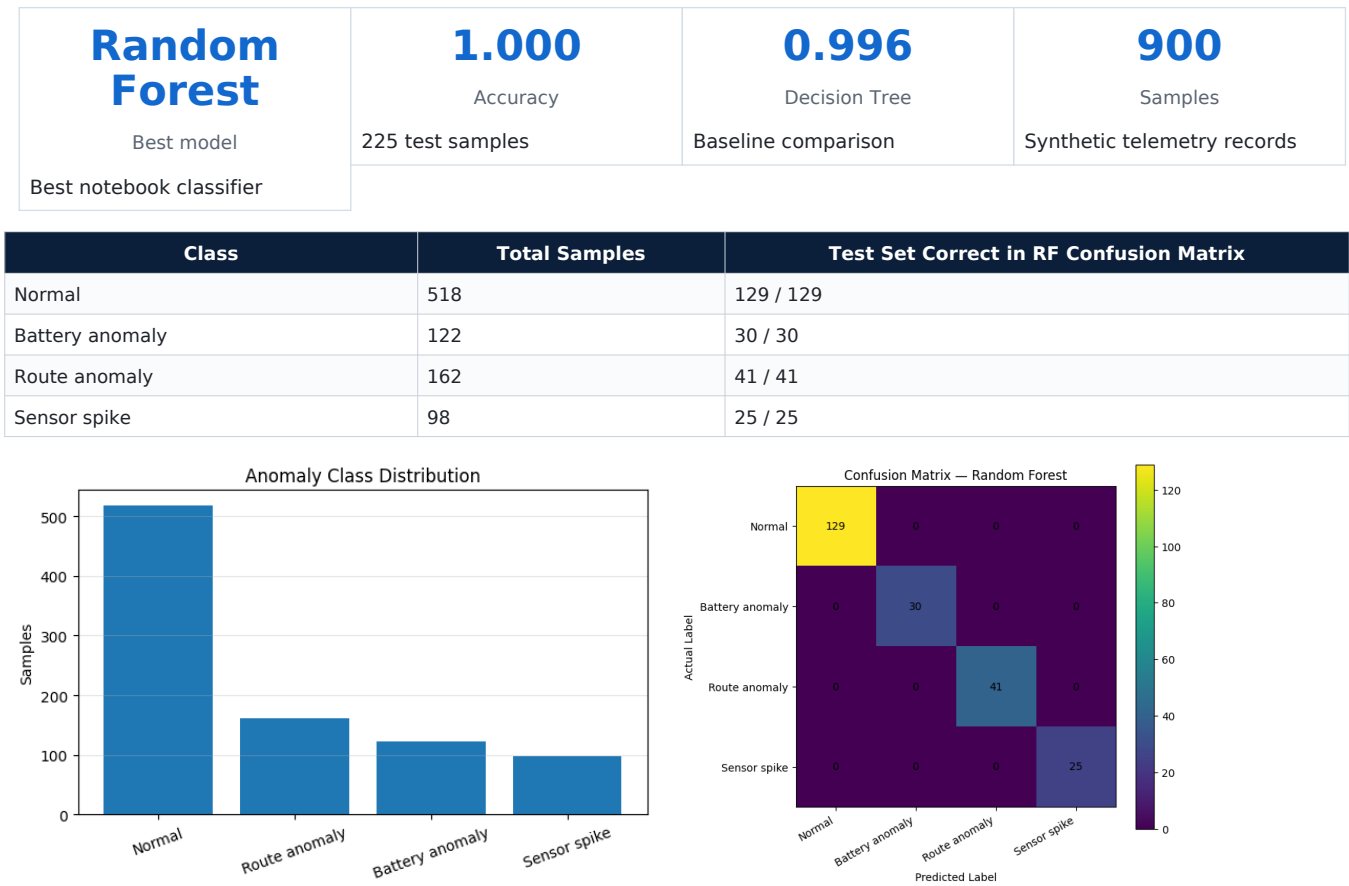
Figure 5: Demand notebook plots. Random Forest predictions are close to actual grouped demand values; zone_density_level, hour, and vehicle_code are the most important predictors.

Honest interpretation

The high R2 is strong for this grouped notebook run, but it should not be oversold as real deployment accuracy. The target was engineered by grouping available delivery records, so the model demonstrates a valid academic forecasting pipeline rather than a production-grade demand system.

9. Notebook Result: Drone Anomaly Classification

The anomaly classifier notebook generated 900 synthetic drone telemetry samples. The class distribution was: Normal = 518, Route anomaly = 162, Battery anomaly = 122, and Sensor spike = 98. Random Forest achieved 1.000 accuracy and Decision Tree achieved 0.996 accuracy on the test split. The confusion matrix shows correct classification for all test samples in the Random Forest run.



Honest interpretation

The anomaly accuracy is almost perfect because labels are rule-generated and the classes are intentionally separable. This is acceptable for a semester simulation, but real UAV telemetry would require noisier data, sensor calibration, and external validation.

10. Integrated ML Interface and 20-Step Simulation

The Streamlit ML panel displays model metrics and a prediction table. The dashboard screenshot proves that the ML pipeline is integrated with the application interface. The notebook results above provide the deeper evidence: dataset loading, model comparison, plots, feature importance, class distribution, and confusion matrix.

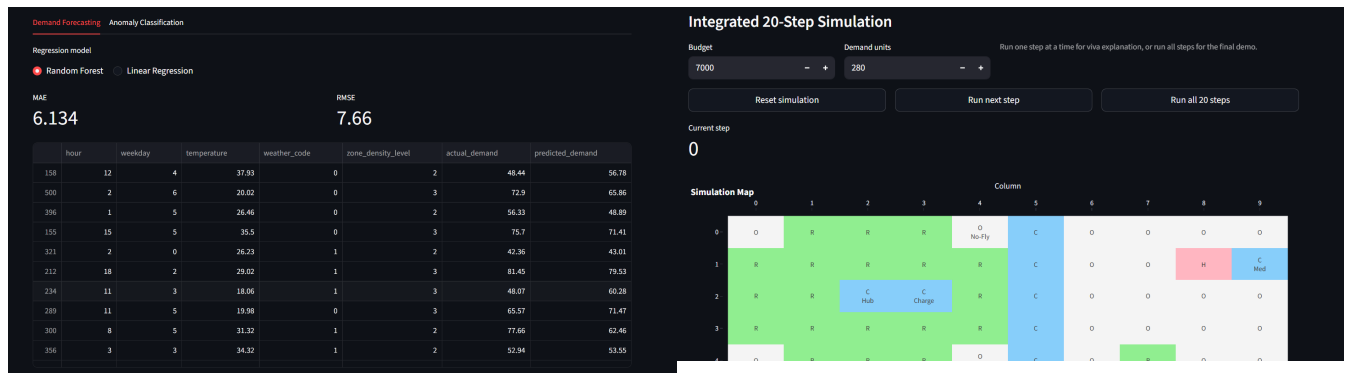


Figure 8: ML pipeline UI and integrated 20-step simulation panel.

The final demo validates the grid, selects the fleet, generates deliveries, computes routes, moves drones, activates a no-fly disruption, reroutes affected drones, runs demand forecasting, detects an anomaly, and prints the final delivery summary. This proves that the modules communicate through one shared model rather than existing as disconnected code files.

11. Conclusion

AeroNet Lite successfully demonstrates CSP, optimization, A* search, dynamic rerouting, regression, and classification in one interactive drone delivery simulator. The updated notebook results strengthen the report by showing actual model outputs from the Amazon Delivery demand notebook and the anomaly classifier notebook.

Limitations: this remains an academic simulator, not a real drone-control platform. It does not model real aerodynamics, GPS uncertainty, air-traffic rules, battery physics, or hardware-level flight control. The anomaly data is synthetic and should be described honestly during viva.

Future improvements: use richer Amazon Delivery feature engineering, add multiple simultaneous drones, include priority-based delivery scheduling, and test anomaly classification with real UAV telemetry such as ALFA UAV fault data.

12. References

1. BSDS Semester Project AI SP2026 project brief: AeroNet Lite.
2. Kaggle Amazon Delivery Dataset by Sujal Suthar - selected dataset for delivery-demand forecasting.
3. Executed notebooks supplied with project: demand_forecasting_amazon_delivery.ipynb and anomaly_classifier.ipynb.